VERIQTA

VERIQTA

Practical. Proven. Production-Ready.

- Production-Ready
- Practical
- Battle Tested
- For DevOps & SREs

WHAT YOU'LL MASTER

- Helm Charts
- Templates
- Values & Configs
- Releases
- Upgrades
- Rollbacks
- Best Practices

HELM PRODUCTION HANDBOOK

VOLUME 1

PAGES 1 – 20

BUILT FOR

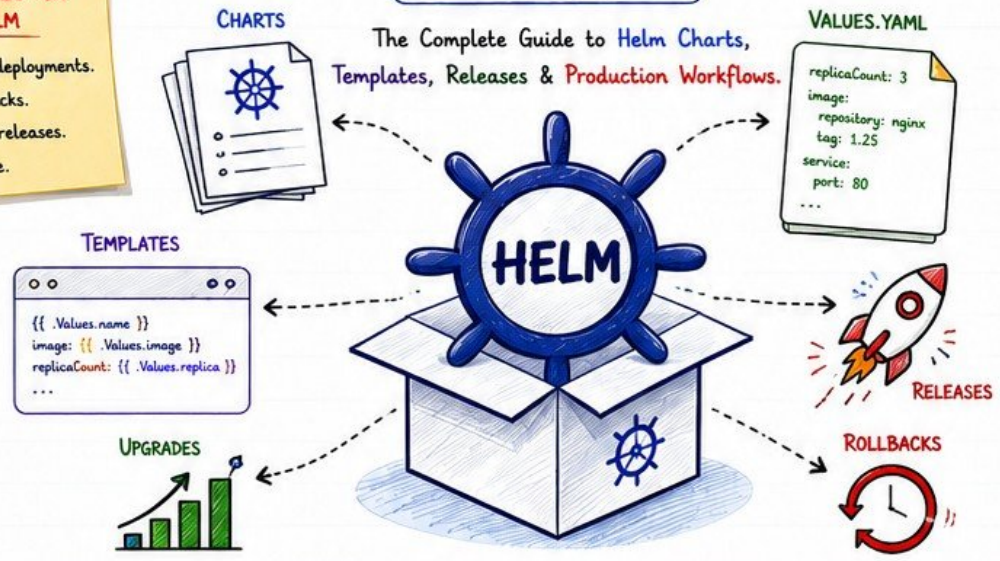
- DevOps Engineers
- SREs
- Platform Engineers

Package.
Deploy.
Scale.
Repeat.
With Helm.

POWERED BY HELM

Consistent deployments.
Easy rollbacks.
Confident releases.
Every time.

The Complete Guide to Helm Charts, Templates, Releases & Production Workflows.



> HELM COMMANDS

```
helm install
helm upgrade
helm rollback
helm template
helm uninstall
...
```

PRODUCTION FOCUSED | SECURE BY DESIGN | AUTOMATE EVERYTHING | OBSERVE & IMPROVE | SIMPLE, NOT EASY

WRITE ONCE. DEPLOY ANYWHERE. SCALE CONFIDENTLY.

Instagram: @veriqta | LinkedIn: VERIQTA | X: @veriqta | YouTube: VERIQTA | VERIQTA

VERIQTA

VERIQTA

HELM PRODUCTION HANDBOOK
VOLUME 1
PAGE 1 OF 20

1. INTRODUCTION TO HELM

"Helm helps you package, deploy, and manage Kubernetes applications the right way – every time."

1 WHAT IS HELM?

Helm is the package manager for Kubernetes. It helps you define, install, and upgrade even the most complex Kubernetes applications.

- ☑ Package your app
- ☑ Manage dependencies
- ☑ Version your releases
- ☑ Roll back with ease
- ☑ Share and reuse

2 WHY HELM EXISTS

- Kubernetes YAML is powerful but verbose.
- Managing apps across environments is hard.
- Keeping configurations DRY is difficult.
- Upgrades and rollbacks are painful manually.
- Helm solves these problems with packaging, templating, and lifecycle management.

Helm brings standardization, reusability, and reliability to Kubernetes deployments.

3 PACKAGE MANAGER FOR KUBERNETES

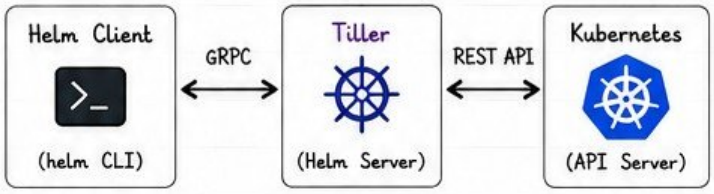
HELM CHART

A Helm chart is a collection of files that describe a related set of Kubernetes resources.

CHART STRUCTURE

```
mychart/
├── Chart.yaml      # Metadata
├── values.yaml     # Default values
├── templates/      # Kubernetes manifests
├── charts/         # Dependencies
└── README.md       # Documentation
```

4 HELM ARCHITECTURE



Helm 3+ : Tiller is removed.
Direct communication between Helm CLI and Kubernetes API.

5 RELEASE LIFECYCLE

NOTE
Every release has a revision history.
You can roll back to any previous good state.

6 PRODUCTION USE CASES

- Deploy complex applications (e.g., Monitoring stack)
- Multi-environment deployments (dev, stage, prod)
- Standardize application deployments
- Share applications across teams
- Version control and audit deployments

EXAMPLES

- Prometheus
- Grafana
- NGINX Ingress
- EFK Stack
- App + DB + Cache

... and many more!

7 HELM VS KUBECTL

HELM	VS	KUBECTL
Package manager		Resource manager
Manages complex apps		Manages individual resources
Templating with values		No templating
Release versioning		No release history
Easy upgrades & rollbacks		Manual and error-prone
Shareable & reusable		Not shareable
Built for application lifecycle		Built for resource operations

8 BEST PRACTICES

- ☑ Follow chart conventions and structure.
- ☑ Keep values.yaml clean and environment-specific.
- ☑ Use meaningful release names.
- ☑ Pin chart versions in production.
- ☑ Test charts in a non-production environment first.
- ☑ Store charts in version control.
- ☑ Use --dry-run and --debug before installing.
- ☑ Monitor releases and keep history.

PRO TIP

Treat Helm charts like code: Review, test, version, and promote with confidence.

Helm = Package. Deploy. Upgrade. Rollback. Repeat.

Helm takes the pain out of Kubernetes application management so you can focus on building, not battling YAML.

VERIQTA
2. INSTALLING HELM

Helm is easy to install on Linux, macOS and Windows.
Choose the method that fits your environment.

1 INSTALLATION METHODS

LINUX
Install using the official script (recommended)

```
$ curl -fsSL https://raw.githubouraccnent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Or using package managers:

- apt install helm (Debian/Ubuntu)
- yum install helm (RHEL/CentOS)
- dnf install helm (Fedora)

macOS
Install using Homebrew (recommended)

```
$ brew install helm
```

Or using the script:

```
$ curl -fsSL https://raw.githubouraccnent.com/helm/helm/main/scripts/get-helm-3 | bash
```

WINDOWS
Use Chocolatey (recommended)

```
> choco install kubernetes-helm
```

Or download the binary:

1. Download from: https://helm.sh/docs/intro/install/
2. Add helm to your PATH

2 VERIFYING INSTALLATION

```
$ helm version
version.BuildInfo{
  Version:"v3.14.2",
  GitCommit:"abcd123",
  GitTreeState:"clean",
  GoVersion:"go1.21.7"
}
```

If you see version info, Helm is installed correctly!

3 HELM CLI OVERVIEW

helm

helm search	Search charts
helm install	Install a chart
helm upgrade	Upgrade a release
helm list	List releases
helm uninstall	Uninstall a release
helm rollback	Rollback a release
helm status	Show release status
helm history	Show release history
helm repo	Manage repositories
helm pull	Download charts
helm template	Render templates locally
helm lint	Lint a chart

4 REPOSITORY CONFIGURATION

```
# Add a repository
$ helm repo add bitnami https://charts.bitnami.com/bitnami
# List repositories
$ helm repo list
# Remove a repository
$ helm repo remove bitnami
```

Repositories store charts for installation.

5 UPDATING REPOSITORIES

```
$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
...Update Complete.
```

Always update before searching or installing charts.

6 HELM ENVIRONMENT

HELM_HOME
~/.cache/helm
(Client configuration)

HELM_REPOSITORY_CACHE
~/.cache/helm/repository
(Repository cache)

HELM_REPOSITORY_CONFIG
~/.config/helm/repositories.yaml
(Repository configuration)

Check Helm environment:

```
$ helm env
HELM_BIN="helm"
HELM_CACHE_HOME="/home/user/.cache/helm"
HELM_CONFIG_HOME="/home/user/.config/helm"
HELM_DATA_HOME="/home/user/.local/share/helm"
HELM_DRIVER="secret"
HELM_NAMESPACE="default"
HELM_PLUGINS="/home/user/.local/share/helm/plugins"
```

7 PRODUCTION SETUP

- Use the latest stable Helm 3 version.
- Install on a bastion or CI/CD runner.
- Store charts in private repositories (e.g., OCI, Artifactory).
- Use version control for custom charts.
- Configure RBAC for Helm operations.
- Use --namespace and --create-namespace in production.
- Enable audit logging for Helm operations.

PRO TIP
Pin chart versions in production to ensure reproducible and predictable deployments.

8 COMMON MISTAKES

- ✖ Not updating repositories before searching.
- ✖ Installing charts without checking versions.
- ✖ Using --force upgrades without understanding impact.
- ✖ Not setting namespaces explicitly.
- ✖ Ignoring chart dependencies.
- ✖ Running Helm without proper permissions (RBAC).
- ✖ Editing released resources manually.

Avoid these to save time and prevent issues in production!

9 QUICK REFERENCE COMMANDS

```
$ helm version
$ helm repo add <name> <url>
$ helm repo update
$ helm search repo nginx
$ helm install myapp bitnami/nginx
$ helm list -A
$ helm upgrade myapp bitnami/nginx
$ helm rollback myapp 1
$ helm uninstall myapp
$ helm status myapp
$ helm history myapp
```

Keep this list handy!

Helm makes Kubernetes easier to manage. Package. Deploy. Upgrade. Repeat.
Automate today. Scale tomorrow. Ship with confidence!

VERIQTA
3. HELM CHARTS

Helm charts are packages of pre-configured Kubernetes resources.
They make applications portable, reusable and easy to manage.

1 WHAT IS A CHART?

A Helm chart is a collection of files that describe a related set of Kubernetes resources.
It defines how to deploy an application to a Kubernetes cluster.

2 CHART STRUCTURE

```
mychart/
├── Chart.yaml
├── values.yaml
├── templates/
├── charts/
├── crds/
├── docs/
├── README.md
└── .helmignore
```

Chart.yaml	Metadata about the chart
values.yaml	Default configuration values
templates/	Kubernetes manifest templates
charts/	Dependent charts (subcharts)
crds/	Custom Resource Definitions
docs/	Documentation
README.md	Chart usage and details
.helmignore	Files to ignore when packaging

3 CHART.YAML

Contains information about the chart.

```yaml
apiVersion: v2                    # Helm chart API version
name: mychart                     # Chart name
description: A sample Helm chart  # Description
type: application                 # Chart type (application/library)
version: 1.0.0                    # Chart version
appVersion: "1.2.3"               # Application version
keywords:                         # Keywords for search
  - nginx
  - web
home: https://veriqta.dev         # Project home
maintainers:                      # Maintainer info
  - name: VERIQTA
    email: admin@veriqta.dev
sources:                          # Upstream sources
  - https://github.com/veriqta/mychart
```

4 VALUES.YAML

The default configuration values for the chart.

```yaml
replicaCount: 3
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "1.25"

service:
  type: ClusterIP
  port: 80

resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 100m
    memory: 128Mi
```

TIP

Override values at install/upgrade time using --set, --values, or --set-file.

5 TEMPLATES/

Contains Kubernetes manifest templates written in YAML with Go templating.

```
templates/
├── _helpers.tpl      # Reusable template snippets
├── deployment.yaml   # Deployment manifest
├── service.yaml      # Service manifest
├── configmap.yaml    # ConfigMap
├── ingress.yaml      # Ingress
└── NOTES.txt         # Info shown after install
```

Templates are rendered with values to produce Kubernetes YAML.

6 CHARTS/ (SUBCHARTS)

Contains dependent charts used by the parent chart.

```
charts/
├── redis/
│   ├── Chart.yaml
│   └── ...
└── mysql/
    ├── Chart.yaml
    └── ...
```

Helm will manage dependencies automatically.

PRODUCTION NOTE

- ☑ Keep charts small, modular and environment-agnostic.
- ☑ Use values for all customization.
- ☑ Test before deploy!

7 PRODUCTION ORGANIZATION

Recommended structure for managing charts at scale.

```
helm-charts/
├── applications/
│   ├── myapp/
│   │   ├── Chart.yaml
│   │   ├── values/
│   │   │   ├── dev.yaml
│   │   │   ├── stage.yaml
│   │   │   └── prod.yaml
│   │   └── templates/ ...
│   └── another-app/
├── libraries/        # Reusable library charts
├── .github/          # CI/CD pipelines
└── README.md         # C...
```

- Environment specific values
- Version control everything
- Automate with CI/CD

8 BEST PRACTICES

- ☑ Follow semantic versioning for charts.
- ☑ Keep values.yaml clean and well documented.
- ☑ Use _helpers.tpl for DRY templates.
- ☑ Avoid hardcoding values in templates.
- ☑ Use .helmignore to keep packages lightweight.
- ☑ Use library charts for shared logic.
- ☑ Test charts with helm lint and helm template.
- ☑ Document all values and usage in README.md.

PRO TIP

Great charts are easy to install, configure and maintain.
Think like a user!

Helm charts = Portable. Reusable. Versioned. Production-Ready.
Build once, deploy anywhere with confidence!

4. Working with Repositories

Helm repositories are collections of charts that you can use, share and manage.

1 Adding Repositories

```
$ helm repo add <name> <url> [flags]
```

Example	Description
$ helm repo add bitnami https://charts.bitnami.com/bitnami	Add Bitnami repo
$ helm repo add stable https://charts.helm.sh/stable	Add stable repo
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts	Add Prometheus repo
$ helm repo list	List all repos

Tip: Give repositories friendly names. They will be used to reference charts (e.g., bitnami/nginx).

2 Searching Charts

```
$ helm search repo <keyword> [flags]
```

Example Command	Description
$ helm search repo nginx	Search charts by keyword
$ helm search repo nginx --versions	Show all versions
$ helm search repo nginx --devel	Include development charts
$ helm search repo bitnami/nginx	Search in a specific repo

Pro Tip: Use --versions to see available versions before installing in production.

3 Repository Updates

```
$ helm repo update
```

- Fetches the latest index.yaml from all repositories.
- Always run before searching or installing charts.
- Helm caches repository data locally.

```
$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
...Successfully got an update from the "prometheus-community" chart repository
Update Complete.
```

4 Removing Repositories

```
$ helm repo remove <name>
```

- ✗ Remove repositories you no longer use.
- ✗ Helps keep your environment clean.
- ✗ Does not delete charts installed from that repo.

Example

```
$ helm repo remove bitnami
```

5 OCI Registries

```
$ helm registry login <registry>
```

- ✓ Helm 3 supports OCI (Open Container Initiative) registries.
- ✓ Store and manage charts like container images.
- ✓ Example: ghcr.io, Amazon ECR, Azure ACR, Harbor

Examples

```
$ helm registrly login ghcr.io
$ helm pull oci://ghcr.io/bitnami/charts/nginx
$ helm install my-nginx oci://ghcr.io/bitnami/charts/nginx
```

6 Private Repositories

```
$ helm repo add private https://charts.mycompany.com
```

- Use authentication (basic auth, token, or certs).
- Configure credentials securely.
- Never hardcode secrets in scripts.

Example (with username/password)

```
$ helm repo add private https://charts.mycompany.com \
  --username $HELM_USER --password $HELM_PASS
```

7 Enterprise Repositories

- ☑ Host internally for security and compliance.
- ☑ Use tools like Harbor, JFrog Artifactory, Nexus.
- ☑ Minror public repos if internet access is restricted.
- ☑ Control access with RBAC and network policies.

Benefits: Security • Reliability • Compliance • Speed

8 Production Recommendations

- ☑ Always run 'helm repo update' in CI/CD before installing.
- ☑ Pin chart versions in production.
- ☑ Use internal/private repositories for reliability.
- ☑ Verify chart provenance and signatures.
- ☑ Limit repository access to trusted teams.
- ☑ Monitor repository availability and backups.

Remember

A reliable repository strategy ensures consistent, secure and repeatable deployments.

Good repositories = Reliable deployments.
Keep them updated, secure and organized!

VERIQTA

5. INSTALLING APPLICATIONS

helm install deploys a chart as a new release into your Kubernetes cluster.

```
$ helm install <release-name> <chart> [flags]
```

1 RELEASE NAMES

- A release is an instance of a chart.
- Names must be unique within a namespace.
- Use meaningful, consistent names.

EXAMPLES

- ☑ payments-api
- ☑ monitoring-stack
- ☑ nginx-ingress
- ☑ frontend-web

2 NAMESPACE DEPLOYMENT

- Deploy to a specific namespace using --namespace.
- If the namespace doesn't exist, create it with --create-namespace.
- Avoid deploying to default in production.

```
$ helm install myapp mychart \
  --namespace production \
  --create-namespace
```

3 CUSTOM VALUES

- Override default values using --values or --set.
- Keep environment-specific values in files.
- Use layered values for complex apps.

```
$ helm install myapp mychart \
  --values values-prod.yaml

$ helm install myapp mychart \
  --set image.tag=1.2.3 \
  --set replicaCount=3
```

4 DRY RUNS

- Preview what will be deployed without applying.
- Great for validation and CI/CD pipelines.
- Use --debug for more details.

```
$ helm install myapp mychart \
  --dry-run

$ helm install myapp mychart \
  --dry-run --debug
```

5 ATOMIC INSTALLS

- Use --atomic to rollback automatically if the install fails.
- Keeps the cluster clean and consistent.
- Great for production safety.

```
$ helm install myapp mychart \
  --atomic
```

6 WAITING FOR RESOURCES

- Use --wait to wait for resources to be ready.
- Use --timeout to set how long to wait.
- Prevents false positives in automation.

```
$ helm install myapp mychart \
  --wait --timeout 10m
```

7 COMBINED BEST PRACTICE

- Combine flags for safe, reliable, observable deployments.
- Recommended for all production installs.

```
$ helm install myapp mychart \
  --namespace prod \
  --create-namespace \
  --values values-prod.yaml \
  --atomic --wait --timeout 10m
```

8 PRODUCTION EXAMPLES

NGINX INGRESS

```
$ helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress \
  --create-namespace \
  --values nginx-values.yaml --atomic --wait
```

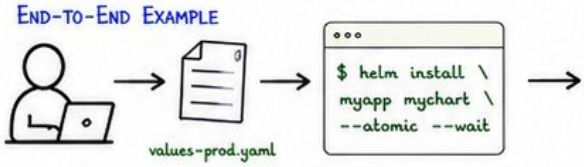
PROMETHEUS STACK

```
$ helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace \
  --values monitoring-values.yaml --atomic --wait
```

APPLICATION

```
$ helm install payments-api mycompany/payments-api \
  --namespace payments \
  --values values-prod.yaml \
  --atomic --wait --timeout 10m
```

END-TO-END EXAMPLE



- ☑ Chart is installed as a release
- ☑ Resources are created in the right namespace
- ☑ Values are applied
- ☑ Cluster waits until everything is ready
- ☑ Rollback automatically on failure

TIP

Always test with --dry-run first!
Always deploy with --atomic --wait in production!

Install with confidence. Deploy with purpose.
Plan. Validate. Install. Verify. Monitor.

6. Values Files

Values files control how your Helm chart behaves.
They make charts flexible and reusable across environments.

1 Values.yaml

The values.yaml file contains default configuration for the chart.

```yaml
replicaCount: 2
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
resources:
  limits:
    cpu: 200m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
ingress:
  enabled: false
  hosts: []
```

YAML

Tip

This file is used as the baseline for all deployments.

2 Default Values

- Every chart comes with sensible defaults.
- They are defined in values.yaml.
- You can install a chart using defaults with no overrides.

```
$ helm install myapp mychart
```

Works out of the box!

3 Overriding Values

Override default values to customize your deployment.

```
$ helm install myapp mychart \
  --values custom-values.yaml
```

Customize to fit your needs.

4 Multiple Values Files

You can specify multiple -f files. Later files override earlier ones.

```
$ helm install myapp mychart \
  -f values.yaml \
  -f values-production.yaml \
  -f values-us-east.yaml
```

Order matters!

5 --set

Override a single value using --set.

```
$ helm install myapp mychart \
  --set replicaCount=3 \
  --set service.type=NodePort
```

Example

Sets replicaCount to 3 and service.type to NodePort

6 --set-string

Use --set-string to force string values.

```
$ helm install myapp mychart \
  --set-string image.tag="01" \
  --set-string env="production" \
```

Example

Ensures the value is treated as a string, not a number or boolean.

7 Environment-Specific Values

Keep separate values files for each environment.

```
mychart/
  environments/
    values-dev.yaml
    values-staging.yaml
    values-prod.yaml
```

- Avoids accidental changes between envs.
- Makes deployments consistent.
- Simplifies CI/CD pipelines.
- Easy to audit and review.

8 Best Practices

- [x] Keep values.yaml minimal and generic.
- [x] Use meaningful default values.
- [x] Use separate files per environment.
- [x] Avoid hardcoding secrets in values files.
- [x] Use --set for small changes, files for big ones.
- [x] Document all custom values.
- [x] Validate changes with --dry-run before deploying.

Pro Tip

A good values structure today prevents outages tomorrow!

Quick Reference

```
helm install myapp mychart                                  # Install with default values
helm install myapp mychart -f values.yaml                   # Use a custom values file
helm install myapp mychart -f a.yaml -f b.yaml              # Multiple values files (last wins)
helm install myapp mychart --set key=value                  # Override single value
helm install myapp mychart --set-string key="value"         # Force string type
helm install myapp mychart --set image.tag=1.25 --set replicaCount=3  # Multiple --set overrides
helm install myapp mychart --dry-run --debug -f values.yaml # Test without installing
```

Always test changes before applying in production!

Good values make great deployments.
Keep them clean. Keep them consistent. Keep them safe.

7. Helm Templates

Helm uses Go templates to render Kubernetes manifests.
You can add logic, reuse code and generate dynamic YAML.

1 Go Template Basics

Helm uses the Go text/template engine.
Delimiters: {{ and }}
Example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-config
data:
  app: {{ .Values.app.name }}
```

Tip Everything inside {{ }} will be evaluated.

2 Variables

Access values from .Values, .Release, .Chart, .Capabilities and more.

```
# Values
{{ .Values.image.repository }}
{{ .Values.service.port }}

# Release
{{ .Release.Name }}
{{ .Release.Namespace }}

# Chart
{{ .Chart.Name }}
{{ .Chart.Version }}

# Capabilities
{{ .Capabilities.KubeVersion.Version }}
```

Root Context The dot (.) represents the current context.

3 Functions

Use built-in functions to transform and format values.

```
# String
{{ upper "hello" }}      --> HELLO
{{ lower "HELLO" }}      --> hello
{{ title "hello world" }} --> Hello World

# Default
{{ .Values.replicaCount | default 3 }}

# Ternary
{{ ternary "yes" "no" .Values.enabled }}

# Math
{{ add 1 2 }}            --> 3
{{ mul 2 3 }}            --> 6
```

4 Pipelines

Pass the output of one function as input to another.

```
# Example
{{ .Values.image.tag | default "latest" | upper }}

# Trim and quote
{{ .Values.name | trim | quote }}

# To YAML (for nested objects)
{{ .Values.resources | toYaml | nindent 2 }}
```

Tip Pipelines make templates clean and readable.

5 Control Structures

Use logic to render content conditionally.

```
# If / Else
{{ if .Values.ingress.enabled }}
  apiVersion: networking.k8s.io/v1
  kind: Ingress
{{ end }}

# With (sets the dot)
{{ with .Values.image }}
  image: {{ .repository }}:{{ .tag }}
{{ end }}

# Range (loop)
{{ range .Values.ports }}
  - port: {{ . }}
{{ end }}
```

IF → WITH → RANGE

6 Includes

Reuse template snippets using include.

```
# Define a template
{{- define "mychart.labels" -}}
  app.kubernetes.io/name: {{ .Chart.Name }}
  app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}

# Use it
metadata:
  labels:
  {{ include "mychart.labels" . | nindent 4 }}
```

Tip Includes keep your templates DRY (Don't Repeat Yourself).

7 Helpers

Store helpers in _helpers.tpl.

```
{{/* Return the full name */}}
{{- define "mychart.fullname" -}}
  {{ printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}
{{- end -}}

{{/* Return chart labels */}}
{{- define "mychart.labels" -}}
helm.sh/chart: {{ printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" }}
{{- end -}}
```

Helpers improve maintainability and consistency across all templates.

8 Production Examples

Conditional Ingress

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "mychart.fullname" . }}
spec:
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "mychart.fullname" . }}
                port:
                  number: {{ .Values.service.port }}
{{- end }}
```

Loop over Ports

```
containers:
- name: {{ .Chart.Name }}
  image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
  ports:
  {{- range .Values.service.ports }}
    - containerPort: {{ . }}
  {{- end }}
```

Tip Templates let Helm adapt to any environment automatically.

Quick Reference Cheat Sheet

Type	Example
Variable	{{ .Values.replicaCount }}
Default	{{ .Values.tag | default "latest" }}
If	{{ if .Values.enabled }} ... {{ end }}
With	{{ with .Values.image }} ... {{ end }}
Range	{{ range .Values.ports }} ... {{ end }}
Include	{{ include "mychart.labels" . }}
Indent	{{ .Values.obj | toYaml | nindent 2 }}

9 Best Practices

- ☑ Keep templates simple and readable.
- ☑ Use helpers and includes to avoid repetition.
- ☑ Validate all values with --dry-run.
- ☑ Use default values for optional settings.
- ☑ Avoid hardcoding values in templates.
- ☑ Use meaingful names for helpers.
- ☑ Document all values in values.yaml.
- ☑ Test templates before deploying to production.

Great templates make great deployments.
Reuse. Simplify. Automate. Deploy with confidence.

VERIQTA

VERIQTA

8. Template Functions

Helm uses Sprig, Go template functions and custom functions to help you transform and format data in your templates.

1 Built-in Functions

Common built-in template functions.

Function	Description
print	Prints values
printf	Prints formatted strings
quote	Wraps value in double quotes
upper	Converts to upper case
lower	Converts to lower case
title	Title cases a string
default	Returns default if value is empty
empty	Returns true if value is empty
coalesce	Returns the first non-empty value

Tip

Use functions to keep templates clean and flexible.

2 Default Values

Use default, coalesce and empty to handle missing values.

```
# default
{{ .Values.replicaCount | default 3 }}

# coalesce (first non-empty)
{{ coalesce .Values.image.tag
        .Chart.AppVersion "latest" }}

# empty
{{ if empty .Values.ingress.host }}
    # no host configured
{{ end }}
```

Best for optional values and fallbacks.

3 String Functions

Work with and transform strings.

Function	Example
upper	{{ upper "hello" }} → "HELLO"
lower	{{ lower "HeLlo" }} → "hello"
title	{{ title "hello world" }} → "Hello World"
trim	{{ trim " hi " }} → "hi"
trimSuffix	{{ trimSuffix ".yaml" "file.yaml" }} → "file"
trimPrefix	{{ trimPrefix "svc-" "svc-nginx" }} → "nginx"
replace	{{ replace "a_b_c" "_" "-" }} → "a-b-c"
contains	{{ contains "prod" .Values.env }}

4 List (Slice) Functions

Work with lists and arrays.

```
# list
{{ $list := list "a" "b" "c" }}

# first, last
{{ first $list }}    → a
{{ last $list }}     → c

# append
{{ $list = append $list "d" }}
{{ $list }}  → [a b c d]

# len
{{ len $list }}  → 4

# join
{{ join "_" $list }}  → "a_b_c"

# has
{{ has "b" $list }}  → true
```

Tip

Lists are zero-indexed but first and last make life easier.

5 Dictionary (Map) Functions

Work with dictionaries (maps).

```
# dict
{{ $m := dict "name" "nginx" "port" 80 }}

# get
{{ get $m "name" }}  → nginx

# set
{{ $_ := set $m "port" 8080 }}
{{ get $m "port" }}  → 8080

# hasKey
{{ hasKey $m "name" }}  → true

# keys, values
{{ keys $m }}    → [name port]
{{ values $m }}  → [nginx 8080]

# merge
{{ merge $m (dict "env" "prod") }}
```

Tip

Maps help you build dynamic YAML blocks.

6 Encoding Functions

Encode or decode data.

Function	Example
b64enc	{{ "hello" | b64enc }} → aGVsbG8=
b64dec	{{ "aGVsbG8=" | b64dec }} → hello
quote	{{ "text" | quote }} → "text"
squote	{{ "text" | squote }} → 'text'
toJson	{{ toJson .Values }} → JSON string
fromJson	{{ fromJson $json }} → map
toYaml	{{ toYaml .Values }} → YAML text

Use encoding for Secrets, ConfigMaps and external integrations.

7 Date & Time Functions

Work with time and dates.

```
# now
{{ now }}
# date
{{ now | date "2006-01-02" }}
→ 2025-07-16
# date with time
{{ now | date "2006-01-02 15:04:05" }}
# unix timestamp
{{ now | unixEpoch }}
# duration
{{ now | dateModify "+24h" }}
```

Tip

Dates are great for annotations, labels and timestamps.

8 Production Usage

Real-world ways these functions help.

- [x] Use default and coalesce for optional values.
- [x] Sanitize and format strings for resource names.
- [x] Work with lists to build ports, hosts and envs.
- [x] Use dictionaries for labels, annotations and configs.
- [x] Encode values for Secrets and ConfigMaps.
- [x] Add timestamps to annotations for traceability.
- [x] Keep templates DRY with helpers + functions.

Example – Dynamic Labels

```
metadata:
  labels:
    app.kubernetes.io/name:  {{ .Chart.Name | lower }}
    app.kubernetes.io/instance:  {{ .Release.Name }}
    app.kubernetes.io/version:  {{ .Chart.AppVersion }}
    build-date:  {{ now | date "2006-01-02" }}
    environment:  {{ .Values.env | default "dev" }}
```

Master template functions. Write smarter templates.
Clean. Flexible. Reusable. Production-Ready.

VERIQTA
9. HELM UPGRADES

Helm upgrades allow you to update a release to a new version or new configuration without downtime.

```
$ helm upgrade <release-name> <chart> [flags]
```

1 HELM UPGRADE

Upgrade an existing release.

```
$ helm upgrade myapp mychart \
  --namespace production \
  --values values-prod.yaml
```

TIP: If the release does not exist, Helm will return an error. Use --install to create if missing.

```
$ helm upgrade --install myapp mychart
```

2 VERSION MANAGEMENT

Control chart and app versions.

```
$ helm search repo mychart --versions
$ helm upgrade myapp myrepo/mychart \
    --version 2.5.1
```

WHAT HELM TRACKS	
☑ Chart Version	(Chart.yaml)
☑ App Version	(values/Deployment)
☑ Release Revision	(Helm release history)

3 SAFE UPGRADES

Always upgrade safely in production.

- Test in non-production first
- Use --dry-run and --debug
- Validate manifests before applying
- Backup important data
- Monitor after upgrade

PRO TIP: Combine --atomic and --wait for the safest upgrade experience.

4 ROLLING UPDATES

Helm upgrades trigger rolling updates (if your resources support it).

Controlled by Kubernetes Deployment strategy.

EXAMPLE – SET ROLLING UPDATE STRATEGY

```
$ helm upgrade myapp mychart \
 --set deployment.strategy.type=RollingUpdate \
 --set deployment.strategy.rollingUpdate.maxUnavailable=25% \
 --set deployment.strategy.rollingUpdate.maxSurge=25%
```

5 DRY-RUN UPGRADES

Preview changes before applying.

```
$ helm upgrade myapp mychart \
  --values values-prod.yaml \
  --dry-run --debug
```

WHAT IT SHOWS YOU

- ☑ Rendered manifests
- ☑ Resources that will change
- ☑ Values being applied
- ☑ Potential errors

6 ATOMIC UPGRADES

Rollback automatically if upgrade fails.

```
$ helm upgrade myapp mychart \
  --values values-prod.yaml \
  --atomic --wait --timeout 10m
```

- ☑ --atomic: rollback on failure
- ☑ --wait: wait for all resources to be ready
- ☑ --timeout: max wait time

This keeps your environment stable even if something goes wrong.

7 PRODUCTION ROLLOUT STRATEGY

1. Test in Dev / Staging
2. Dry Run & Validate
3. Upgrade in Canary 10%
4. Monitor & Verify
5. Full Rollout

CANARY EXAMPLE

```
$ helm upgrade myapp mychart \
  --set ingress.annotations."nginx.ingress.kubernetes.io/canary"="true" \
  --set ingress.annotations."nginx.ingress.kubernetes.io/canary-weight"="10"
```

BLUE/GREEN EXAMPLE

```
$ helm upgrade myapp-blue mychart -f values-blue.yaml
(test traffic on blue)
$ helm upgrade myapp-green mychart -f values-green.yaml
(switch traffic to green)
$ helm uninstall myapp-blue
```

Always have a rollback plan before full rollout.

8 COMMON MISTAKES

- ✕ Upgrading without testing in staging.
- ✕ Not using --atomic or --wait.
- ✕ Ignoring breaking changes in new versions.
- ✕ Skipping backups for stateful workloads.
- ✕ Overriding critical values accidentally.
- ✕ Not monitoring after upgrade.
- ✕ Using --force (unless absolutely needed).

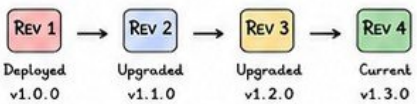
WHY IT MATTERS

One wrong upgrade can cause downtime, data loss, or application instability.

9 USEFUL UPGRADE COMMANDS

COMMAND	PURPOSE
`$ helm upgrade <release> <chart>`	Upgrade a release
`$ helm upgrade --install <release> <chart>`	Upgrade or install if not exists
`$ helm upgrade --dry-run --debug`	Preview changes
`$ helm upgrade --atomic --wait`	Safe upgrade with rollback and wait
`$ helm history <release>`	View release revision history
`$ helm rollback <release> <revision>`	Rollback to a previous revision

RELEASE REVISION FLOW



QUICK CHECKLIST

- ✓ Test before upgrade
- ✓ Dry run first
- ✓ Use atomic + wait
- ✓ Monitor after upgrade
- ✓ Have rollback plan

Upgrades should bring value, not problems.
Plan it. Test it. Roll it out. Monitor it. Succeed!

10. Rollback and Release Management

Helm keeps a full history of every release.
You can rollback to any previous working state quickly and safely.

```
$ helm rollback <release-name> <revision> [flags]
```

1 Helm Rollback

Rollback to a previous revision.

```
$ helm rollback myapp 3
```

Common Flags

--dry-run	Preview rollback
--wait	Wait for resources
--atomic	Rollback on failure
--cleanup-on-fail	Clean failed release
--history-max	Set history limit

Tip
If no revision is specified, Helm rolls back to the previous revision.

2 Release History

View full history of a release.

```
$ helm history myapp
```

Rev	Updated	Status	Chart	App Version	Description
6	2025-07-16 14:22	deployed	myapp-2.6.0	1.3.0	Upgrade complete
5	2025-07-16 13:45	failed	myapp-2.6.0	1.3.0	Upgrade failed
4	2025-07-16 12:30	deployed	myapp-2.5.0	1.2.0	Upgrade complete
3	2025-07-15 10:10	deployed	myapp-2.4.0	1.1.0	Upgrade complete
2	2025-07-14 09:00	deployed	myapp-2.3.0	1.0.0	Install complete
1	2025-07-14 08:45	superseded	myapp-2.3.0	1.0.0	Initial install

Status key: deployed = success, failed = failed, superseded = replaced

3 Revision Numbers

Every change creates a new revision.

- Install = revision 1
- Upgrade = revision +1
- Rollback = new revision
- No gaps in numbers
- History is immutable

Example

```
Install          --> rev 1
Upgrade          --> rev 2
Upgrade (fail)   --> rev 3
Rollback to 1    --> rev 4
```

4 Failed Releases

When an upgrade fails, Helm records it.

- Status = failed
- Resources may be in partial state
- Pods may be crashing
- Services may be broken

Check failed revision details:

```
$ helm history myapp
$ helm get manifest myapp --revision 5
$ helm get values   myapp --revision 5
```

5 Recovery Workflow

1. Detect — Monitor alerts or user reports
2. Investigate — Check history, logs & events
3. Decide — Fix forward or rollback
4. Execute — Rollback or hotfix deploy
5. Validate — Confirm system is healthy

Useful Commands

```
$ helm status myapp
$ helm history myapp
$ helm get values myapp --revision 3
$ helm get manifest myapp --revision 3
$ helm rollback myapp 3 --atomic --wait
```

6 Rollback Strategy

Choose the right rollback approach.

1. Rollback to last known good revision
2. Validate after rollback
3. If same issue persists, rollback further
4. If config issue, use values from older rev
5. Communicate and monitor closely

Always rollback to a tested, stable revision.

7 Production Recovery

Example: Upgrade broke the app.

Incident Detected → Check History → Rollback to Stable Rev → Validate System → Monitor Closely

Example Timeline

```
13:45  Upgrade to rev 5 --> failed
13:50  Rollback to rev 4 --> deployed
13:52  Validation --> success
14:00  Monitoring --> normal
```

8 Best Practices

- [x] Always test upgrades in staging first.
- [x] Use --atomic and --wait for safe upgrades.
- [x] Keep history-max high enough (e.g., 20+).
- [x] Tag known good revisions in documentation.
- [x] Monitor after every upgrade or rollback.
- [x] Automate rollback for critical services when possible.
- [x] Back up values files and custom configs.
- [x] Perform regular disaster recovery drills.

Tip
A good rollback plan is cheaper than a long outage.

9 Common Mistakes

- ✗ Rolling back without checking root cause
- ✗ Using wrong revision number
- ✗ Not using --atomic or --wait
- ✗ Ignoring partial resource cleanup
- ✗ Not validating after rollback
- ✗ Deleting release history
- ✗ Overwriting values without backups

Warning
Rollback is powerful but not a substitute for understanding the issue.

10 Release Life Cycle

Install (Create rev 1) → Upgrade (Create rev 2) → Upgrade (Create rev 3) → Failed Upgrade (Create rev 4 (failed)) → Rollback (Create rev 5) → Stable (System Healthy)

Remember
Every release action creates a revision.
History is your safety net.
Use it wisely!

Good teams ship fast. Great teams recover fast.
Plan. Test. Monitor. Rollback. Repeat.

VERIQTA
11. DEPENDENCIES

Helm charts can depend on other charts.
This makes complex applications modular, reusable and easy to manage.

Parent chart → pulls → Child charts → builds → Complete app

1 PARENT CHARTS

The main chart that depends on other charts.

```
myapp/ (Parent Chart)
├── Chart.yaml
├── values.yaml
├── charts/
│   ├── redis/
│   ├── postgresql/
│   └── nginx/
└── templates/
```

TIP
Parent charts manage values, configuration and how child charts are used.

2 CHILD CHARTS

Reusable, standalone charts.

EXAMPLE CHILD CHARTS

- ☑ redis
- ☑ postgresql
- ☑ nginx-ingress
- ☑ prometheus
- ☑ grafana

Build once, use anywhere.

3 CHART DEPENDENCIES

Declare dependencies in Chart.yaml

```yaml
dependencies:
- name: redis
  version: 17.13.0
  repository: https://charts.bitnami.com/bitnami
- name: postgresql
  version: 15.5.20
  repository: https://charts.bitnami.com/bitnami
- name: nginx-ingress
  version: 4.10.0
  repository: https://kubernetes.github.io/ingress-nginx
```

Each dependency has name, version and repository.

4 DEPENDENCY UPDATE

Update dependencies defined in Chart.yaml.

```
$ helm dependency update
```

- ☑ Downloads charts into the charts/ directory
- ☑ Respects version constraints
- ☑ Updates Chart.lock automatically

NOTE
Run this command after changing Chart.yaml or version constraints.

5 DEPENDENCY BUILD

Package dependencies from local charts/.

```
$ helm dependency build
```

- ☑ Reads Chart.lock
- ☑ Packages local dependencies
- ☑ Reconstructs charts/ from lock file
- ☑ Useful in CI/CD environments

TIP Use build in pipelines for reproducible builds.

6 VERSION CONSTRAINTS

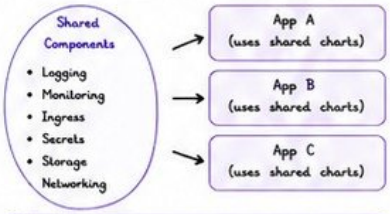
Control which versions can be used.

CONSTRAINT	MEANING
=1.2.3	Exactly version 1.2.3
~1.2.0	1.2.x >=1.2.0 and <1.3.0
^1.2.0	>=1.2.0 and <2.0.0
>=1.2.0	Any version >=1.2.0
>1.2.0, <2.0.0	Between 1.2.0 and 2.0.0
*	Any version (not recommended)

TIP Use ^ for most production charts.

7 SHARED COMPONENTS

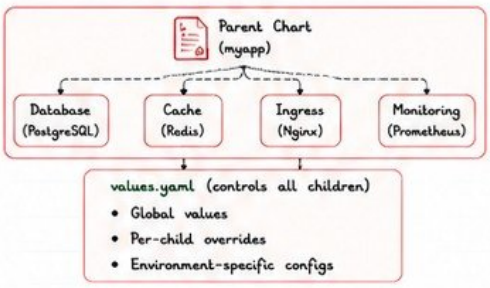
Extract common services into reusable charts.



Shared components reduce duplication and improve consistency.

8 PRODUCTION DESIGN

Recommended dependency structure.

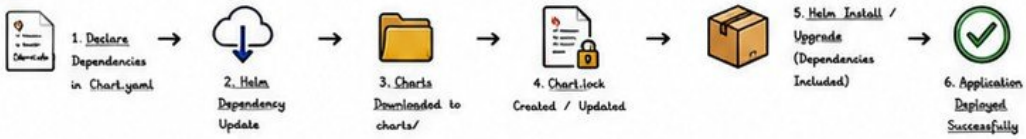


9 BEST PRACTICES

- ☑ Keep child charts simple and focused.
- ☑ Pin versions using ^ or ~.
- ☑ Regularly run helm dependency update.
- ☑ Commit Chart.lock to version control.
- ☑ Use shared charts across projects.
- ☑ Avoid wildcard (*) versions.
- ☑ Document all dependencies.
- ☑ Test charts after dependency updates.
- ☑ Use private repos for internal charts.

TIP Treat dependencies like code. Version, test and validate them.

10 DEPENDENCY WORKFLOW



COMMON COMMANDS

```
$ helm dependency update  # fetch latest deps
$ helm dependency build   # build from lock file
$ helm dependency list    # list dependencies
$ helm dependency status  # check status
```

Good dependencies make strong applications.
Modular. Reusable. Reliable. Production-Ready.

12. Helm Hooks

Helm hooks let you run Kubernetes resources at specific points in a release lifecycle.

```
# Hooks are annotations on Kubernetes resources
metadata:
  annotations:
    "helm.sh/hook": pre-install
```

1 Pre-Install

Runs before any resources are installed.

Use cases
- Database migrations
- Secrets creation
- Prerequisite checks
- External system setup

```
annotations:
  helm.sh/hook: pre-install
  helm.sh/hook-weight: "0"
```

2 Post-Install

Runs after all resources are installed.

Use cases
- Smoke tests
- Notifications
- Data seeding
- Setup completion tasks

```
annotations:
  helm.sh/hook: post-install
  helm.sh/hook-weight: "0"
```

3 Pre-Upgrade

Runs before an upgrade is applied.

Use cases
- Backup data
- Pre-upgrade validation
- Maintenance mode
- Schema migrations

```
annotations:
  helm.sh/hook: pre-upgrade
  helm.sh/hook-weight: "0"
```

4 Post-Upgrade

Runs after an upgrade completes successfully.

Use cases
- Verify deployment
- Cache warmup
- Notifications
- Post-upgrade tests

```
annotations:
  helm.sh/hook: pre-upgrade
  helm.sh/hook-weight: "0"
```

5 Pre-Delete

Runs before resources are deleted.

Use cases
- Backup before delete
- Deregister services
- Drain traffic
- External cleanup

```
annotations:
  helm.sh/hook: pre-delete
  helm.sh/hook-weight: "0"
```

6 Hook Weights

Control the order in which hooks run.

Weight	Order
-10	Runs first
-5	Runs before weight 0
0	Default weight
5	Runs after weight 0
10	Runs last

Tip

Lower weights run first. Same weight = alphabetical order.

```
annotations:
  helm.sh/hook: pre-install
  helm.sh/hook-weight: "-5"
```

7 Hook Deletion Policy

Control how Helm cleans up hook resources.

Policy	Description
before-hook-creation	Delete previous hook resource before creating a new one.
hook-succeeded	Delete hook resource after it succeeds.
hook-failed	Delete hook resource after it fails.
hook-succeeded,hook-failed	Delete in both success and failure cases.
keep	Never delete hook resources.

Tip

Use deletion policy to avoid resource accumulation.

```
annotations:
  helm.sh/hook: pre-install
  helm.sh/hook-delete-policy:
    - before-hook-creation
    - hook-succeeded
```

8 Production Examples

Database Migration (Pre-Install)

Run migrations before app is installed.

```
annotations:
  helm.sh/hook: pre-install
  helm.sh/hook-weight: "-10"
  helm.sh/hook-delete-policy:
    - before-hook-creation
    - hook-succeeded
```

Backup Before Upgrade (Pre-Upgrade)

Backup data before upgrade.

```
annotations:
  helm.sh/hook: pre-upgrade
  helm.sh/hook-weight: "-5"
  helm.sh/hook-delete-policy:
    - before-hook-creation
    - hook-succeeded
```

Smoke Test After Upgrade (Post-Upgrade)

Ensure new version is healthy.

```
annotations:
  helm.sh/hook: post-upgrade
  helm.sh/hook-weight: "5"
  helm.sh/hook-delete-policy:
    - hook-succeeded
    - hook-failed
```

Cache Warmup (Post-Install)

Warm up cache for better performance.

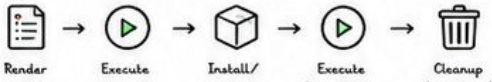
```
annotations:
  helm.sh/hook: post-install
  helm.sh/hook-weight: "10"
  helm.sh/hook-delete-policy:
    - hook-succeeded
```

Cleanup Before Delete (Pre-Delete)

Remove external resources before deleting app.

```
annotations:
  helm.sh/hook: pre-delete
  helm.sh/hook-weight: "0"
  helm.sh/hook-delete-policy:
    - hook-succeeded
```

Hook Execution Lifecycle



Best Practices

- [x] Use hooks only for tasks Kubernetes cannot manage.
- [x] Keep hooks idempotent (safe to run multiple times).
- [x] Set appropriate weights for correct ordering.
- [x] Use deletion policies to prevent resource clutter.
- [x] Monitor hook Jobs and handle failures.

Common Mistakes

- ✗ Using hooks for core app logic.
- ✗ Not setting hook weights.
- ✗ Forgetting deletion policy.
- ✗ Ignoring failed hook Jobs.
- ✗ Relying on hooks for availability.

Hooks are powerful. Use them wisely to automate the right things at the right time.
Automate with hooks. Deploy with confidence. Operate like a pro.

13. Secrets and ConfigMaps

Store configuration and sensitive data outside your images.
Use ConfigMaps for non-sensitive data and Secrets for sensitive data.

Externalize → Configure → Secure → Inject

1 Managing ConfigMaps

Store non-sensitive configuration.

Create ConfigMap

```
# From file
kubectl create configmap app-config \
  --from-file=config.yaml

# From literal
kubectl create configmap app-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_X=true
```

Use in Pod (values.yaml)

```
envFrom:
  - configMapRef:
      name: app-config
```

2 Managing Secrets

Store sensitive data securely.

Create Secret

```
# Generic secret (file)
kubectl create secret generic app-secret \
  --from-file=secret.yaml

# Inline (base64 encoded automatically)
kubectl create secret generic app-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASS="P@ssw0rd"
```

Use in Pod (values.yaml)

```
envFrom:
  - secretRef:
      name: app-secret
```

Secrets are base64 encoded in Kubernetes. Anyone with cluster access can decode them. Encrypt at rest!

3 Template Secrets

Reference secrets in Helm templates.

values.yaml

```
secret:
  name: app-secret
```

deployment.yaml (template)

```
env:
  - name: DB_USER
    valueFrom:
      secretKeyRef:
        name: {{ .Values.secret.name }}
        key: DB_USER

  - name: DB_PASS
    valueFrom:
      secretKeyRef:
        name: {{ .Values.secret.name }}
        key: DB_PASS
```

Use .Values to keep templates flexible and reusable.

4 Sensitive Values

Never hardcode sensitive data.

Bad Practice	Good Practice
password: "mypassword"	Use Secrets
apiKey: "123456"	Use Secrets
token: " abcd..."	Use Secrets

Never commit secrets to Git!

Sensitive Data Examples

- [x] Passwords
- [x] API Keys
- [x] Tokens
- [x] Certificates
- [x] Private Keys
- [x] Database Credentials

5 Secret Management

Handle secret lifecycle securely.

- [x] Create with least privilege
- [x] Rotate regularly
- [x] Avoid long-lived credentials
- [x] Audit access and usage
- [x] Limit secret size (< 1MB)

Tip
Automate secret rotation where possible (Key rotation, DB rotation, etc).

6 Security Considerations

Protect secrets at every layer.

- [x] Enable RBAC - restrict who can read secrets
- [x] Enable encryption at rest in Kubernetes
- [x] Use NetworkPolicies to limit access
- [x] Scan images for leaked secrets
- [x] Avoid printing secrets in logs
- [x] Use least privilege service accounts
- [x] Review secrets in CI/CD pipelines

Anyone with 'get secrets' permission can read all secrets in that namespace.

7 External Secrets

Integrate with external secret stores.

External Secret Store
- AWS Secrets Manager
- HashiCorp Vault
- Azure Key Vault
- Google Secret Manager

→ External Secrets Operator → Kubernetes Secret

External Secrets Operator syncs secrets from external stores into Kubernetes automatically.

8 Best Practices

- [x] Use ConfigMaps for non-sensitive data only
- [x] Use Secrets for all sensitive data
- [x] Never hardcode values in templates
- [x] Use values.yaml placeholders
- [x] Encrypt secrets at rest
- [x] Rotate secrets regularly
- [x] Use external secret stores in production
- [x] Validate secret existence in CI/CD
- [x] Document secret usage and ownership

9 Production Examples

Application Config (ConfigMap)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  FEATURE_X: "true"
  API_URL: "https://api.dev"
```

Used for:
- ✓ App settings
- ✓ Feature flags
- ✓ URLs, endpoints

Database Credentials (Secret)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  type: Opaque
data:
  username: YWRtaW4=
  password: UEBac3cwcmQ=
```

Used for:
- ✓ DB user/password
- ✓ Connection strings
- ✓ Encrypted values

TLS Certificates (Secret)

```
apiVersion: v1
kind: Secret
metadata:
  name: tls-cert
  type: kubernetes.io/tls
data:
  tls.crt: LS0tLS1CRUdJTi...
  tls.key: LS0tLS1CRUdJTi...
```

Used for:
- ✓ TLS certificates
- ✓ Private keys
- ✓ HTTPS encryption

Third-Party API Keys (Secret)

```
apiVersion: v1
kind: Secret
metadata:
  name: api-keys
  type: Opaque
data:
  stripe_key: c2tfdGVzdC4uLg==
  sendgrid_key: UOGMeS4.uLg==
```

Used for:
- ✓ API keys/tokens
- ✓ Service credentials
- ✓ External integrations

Protect your secrets. Protect your systems. Protect your customers.
Secure by design. Manage with care. Trust with discipline.

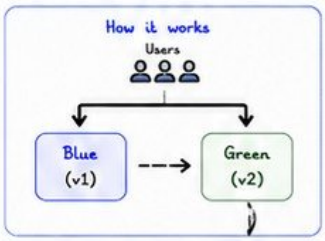
14. Production Deployments

Helm + Kubernetes make it easy to deploy.
But production needs strategies that protect users and business.

Safe Delivery ⇒ Small Changes + Fast feedback + Easy Rollback

1 Blue-Green Deployment

Two environments. Switch traffic only when new version is ready.

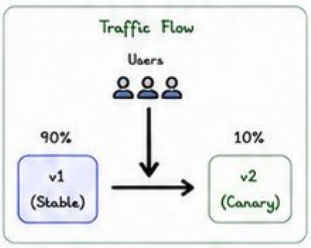


- ☑ Deploy new version to Green
- ☑ Test Green environment
- ☑ Switch traffic from Blue to Green
- ☑ Keep Blue as rollback option
- ☑ Delete old environment later

Best for:
Major releases with high risk tolerance

2 Canary Deployment

Release to a small % of users first. Validate, then increase gradually.

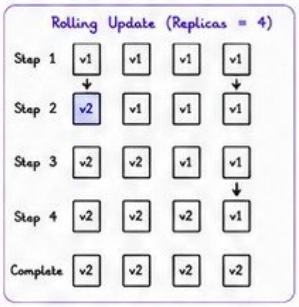


- ☑ Deploy new version (small %)
- ☑ Monitor metrics and errors
- ☑ Increase % gradually (10% → 25% → 50% → 100%)
- ☑ Rollback if issues detected

Best for:
High-traffic apps, continuous delivery

3 Rolling Updates

Kubernetes replaces pods gradually to avoid downtime.



- ☑ Uses maxUnavailable & maxSurge
- ☑ Keeps app available during rollout
- ☑ Default and safest strategy
- ☑ Works great with Deployments

4 Zero Downtime Goal

No user should see errors or downtime.

Key Principles

- ☑ Deploy small and frequently
- ☑ Automate testing
- ☑ Health checks must be accurate
- ☑ Use readiness gates
- ☑ Monitor everything
- ☑ Have instant rollback plan

Remember

Zero downtime is not a feature.
It is an engineering discipline.
Plan for it. Test for it.
Automate it.

5 Health Checks

Detect problems early. Remove unhealthy pods from traffic.

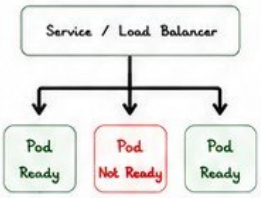
Types

- Liveness: Is the app alive?
- Readiness: Is the app ready to serve?
- Startup: Does the app need time to start?
- Custom: Business or endpoint checks

Good health checks prevent bad traffic and cascading failures.

6 Readiness Probes

Ensure pods are ready before receiving traffic.



Example (HTTP Probe)

- Path: /health
- Port: 8080
- Initial Delay: 10s
- Period: 10s
- Timeout: 2s
- Success Threshold: 1
- Failure Threshold: 3

Readiness = Protect users.
Liveness = Protect infrastructure.

7 Rollback Planning

Always have a quick and tested rollback path.

Rollback Plan Checklist

- ☑ Keep previous version available
- ☑ Test rollback process regularly
- ☑ Use helm rollback or previous chart
- ☑ Have database migration rollback plan
- ☑ Monitor after rollback
- ☑ Document and train the team

A rollback you haven't tested will fail when you need it most.

8 Production Deployment Workflow

Deployment Strategy Comparison

Strategy	Downtime	Risk	Complexity	Best For
Blue-Green	None	Low	Medium	Big releases
Canary	None	Very Low	High	High traffic apps
Rolling Update	None	Low	Low	Most workloads

Helm Commands for Deployment

```
$ helm upgrade --install myapp ./chart \
  --namespace prod --atomic --wait --timeout 10m
$ helm history myapp
$ helm rollback myapp <revision>
$ helm status myapp
$ helm diff upgrade myapp ./chart
```

Production Checklist

- ☑ Values reviewed and tested
- ☑ Health checks configured
- ☑ Resources (CPU/Memory) set
- ☑ Alerts and dashboards ready
- ☑ Rollback plan tested
- ☑ Team notified

Common Mistakes

- ✗ No health checks
- ✗ Deploying untested changes
- ✗ Using --force in production
- ✗ Ignoring readiness probes
- ✗ No rollback plan
- ✗ Not monitoring after deploy

Great deployments are boring. And boring is perfect.
Plan well. Test always. Deploy safely. Monitor constantly. Rollback fast.

VERIQTA

15. Helm Troubleshooting

Problems will happen. Senior engineers debug fast and fix smart.

Observe → Investigate → Isolate → Fix → Verify

Most Helm issues are simple once you know how to check the right things.

1 Debugging Charts

Validate and lint your chart before deploying.

Useful Commands

```
$ helm lint ./mychart
$ helm template myapp ./mychart
$ helm template myapp ./mychart -f values.yaml
$ helm template myapp ./mychart --debug
$ helm show chart ./mychart
$ helm show values ./mychart
$ helm show all myrepo/mychart
```

Tip
Use "helm template" first.
If it's wrong there, it will fail in the cluster.

2 Template Rendering

See the final YAML that Helm will apply.

Examples

```
$ helm template myapp ./mychart > output.yaml
$ helm template myapp ./mychart \
  -f values-prod.yaml --debug
$ helm template myapp ./mychart | less
```

Check For:

- ☑ Wrong indentation
- ☑ Nil pointer errors
- ☑ Missing values
- ☑ Invalid YAML
- ☑ Wrong resource names

3 Failed Installations

The release did not install successfully.

Investigation Steps

1. Check release status
2. Check Events
3. Check Pod status
4. Check Logs
5. Check values and templates

Commands

```
$ helm status myapp
$ helm get all myapp
$ kubectl get events -A
$ kubectl get pods -A
$ kubectl logs <pod-name> -n <ns>
```

4 Upgrade Failures

Upgrades can fail for many reasons.

Common Causes

- ✗ Breaking changes in values
- ✗ Immutable field changes
- ✗ Hook failures
- ✗ Resource conflicts
- ✗ Bad templates
- ✗ Readiness/Liveness failures

Debug Commands

```
$ helm upgrade myapp ./mychart --debug
$ helm upgrade myapp ./mychart -f values.yaml --dry-run
$ helm history myapp
$ helm get values myapp -a
```

Tip
Always test upgrades in stage first!

5 Release Conflicts

Conflicts happen when a release or resource already exists.

Common Issues

- ✗ Release name already exists
- ✗ Resource already exists (CRDs, Namespaces)
- ✗ Manual resource conflict with Helm
- ✗ Different release trying to manage same resource

Resolution

- ☑ Use a unique release name
- ☑ Import existing resources with care
- ☑ Use "helm uninstall" if safe
- ☑ Check ownership labels:
  app.kubernetes.io/managed-by: Helm

Helm manages ownership through labels and annotations. Do not remove them.

6 Kubernetes Events

Events tell the real story.

Commands

```
$ kubectl get events -A --sort-by=.lastTimestamp
$ kubectl get events -n <namespace>
$ kubectl describe pod <pod> -n <namespace>
$ kubectl describe deployment <name> -n <ns>
```

Look For:

- ☑ FailedMount
- ☑ ImagePullBackOff
- ☑ CrashLoopBackOff
- ☑ Readiness probe failed
- ☑ Permission denied
- ☑ Out of memory

7 Common Errors

Values Errors
- nil pointer evaluating interface {}"
- missing value
- wrong data type

Fix: Check values.yaml and default values.

Template Errors
- unexpected EOF
- unexpected {{ end }}
- bad indentation
- invalid YAML

Fix: Use "helm template --debug".

Hook Errors
- pre-install hook failed
- post-upgrade hook failed
- jobs not completing

Fix: Check hook logs and job status.

Resource Errors
- already exists
- forbidden
- immutable field
- quota exceeded

Fix: Check permissions, limits and existing objects.

Upgrade Errors
- cannot patch resource
- context deadline exceeded
- timed out waiting for condition

Fix: Increase timeout, check cluster load.

Install Errors
- failed to install release: another operation in progress
- rendered manifests contain a resource that already exists

Fix: Wait or clean up conflicting release.

8 Production Debugging Workflow

1. **Observe** — What is failing? Collect symptoms.
2. **Investigate** — Check status, events, logs.
3. **Isolate** — Find the root cause.
4. **Fix** — Fix values, templates or configuration.
5. **Verify** — Re-deploy in stage. Validate.
6. **Deploy** — Deploy to prod with confidence.

9 Best Practices

- ☑ Always lint and template before deploy.
- ☑ Use --debug to get more details.
- ☑ Check Events first in any failure.
- ☑ Keep charts simple and modular.
- ☑ Use meaningful release names.
- ☑ Test upgrades in a staging environment.
- ☑ Keep rollback plan ready.
- ☑ Document issues and solutions.

Great engineers prevent problems. Senior engineers solve them fast.
Observe well. Debug fast. Fix smart. Deliver with confidence.

VERIQTA

16. Helm Security

Helm Production Handbook
Volume 1
Page 16 of 20

Security is not a feature. It is a responsibility.
Secure charts. Secure deployments. Secure everything.

Secure by design. Verify everything. Trust nothing by default.

1 Chart Verification

Always verify charts before installing.

Verify Remote Chart

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx
helm pull bitnami/nginx --version 15.9.0
helm verify nginx-15.9.0.tgz
```

Checksum Validation

- [x] Verifies chart integrity
- [x] Prevents tampered packages

2 Provenance Files

Provenance (.prov) files prove chart origin.

Provenance File Contains:

- Chart metadata
- Maintainer info
- Digest (SHA256)
- Signature
- Version

Verify Provenance

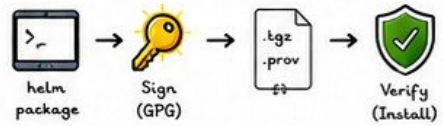
```
helm verify nginx-15.9.0.tgz
--keyring k8s-keyring.gpg
```

Only install charts with valid provenance.

3 Signed Charts

Use GPG signatures to trust chart publishers.

Signing Workflow



Benefits

- [x] Ensures authenticity
- [x] Prevents supply chain attacks
- [x] Builds trust in production pipelines

4 RBAC For Helm

Limit who can install, upgrade, and delete releases.

Example: Helm Release Manager Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: helm-release-manager
rules:
- apiGroups: [""]
  resources: ["secrets", "configmaps"]
  verbs: ["get", "list", "watch", "create", "update",
  "patch", "delete"]
- apiGroups: ["apps"]
  resources: ["deployments", "statefulsets"]
  verbs: ["get", "list", "watch", "update", "patch"]
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "watch"]
```

Never use cluster-admin for Helm operations.
Give least privilege access.

5 Secret Protection

Secrets should never be exposed in charts.

Bad Practice

```
values.yaml
mysql:
  password: "MySuperSecret123"
```

Good Practice

```
values.yaml
mysql:
  password: ""  # use external secret
  existingSecret: mysql-secret
```

Use Kubernetes Secrets

```
kubectl create secret generic mysql-secret \
  --from-literal=password='MySuperSecret123'
```

Never commit secrets to Git!
Use external secret stores.

6 Least Privilege

Grant only the permissions required.

Principles

- [x] No cluster-admin
- [x] No wildcard permissions (*)
- [x] Scope to namespace
- [x] Limit API groups and resources
- [x] Review permissions regularly

Check Access

```
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa
kubectl auth can-i "*" "*" --as=system:serviceaccount:ns:sa
```

7 Image Security

Use secure, trusted container images.

Best Practices

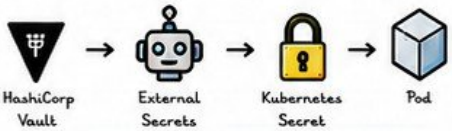
- [x] Use official or trusted registries
- [x] Scan images for vulnerabilities
- [x] Pin image by digest, not tag
- [x] Run as non-root user
- [x] Use read-only file systems
- [x] Drop unnecessary capabilities

Use digests:

```
image: nginx@sha256:3f2a....b97c
```

8 Enterprise Secret Management

Integrate Helm with external secret stores.



Integration Options

- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- External Secrets Operator
- SOPS + Helm

9 Enterprise Security Checklist

- [x] Verify chart integrity and provenance
- [x] Use signed charts from trusted sources
- [x] Enforce RBAC and least privilege
- [x] Protect secrets with external stores
- [x] Scan container images and dependencies
- [x] Use network policies
- [x] Enable audit logging
- [x] Review and rotate credentials
- [x] Automate security in CI/CD
- [x] Continuously monitor and alert

Production Security Workflow

Security is everyone's responsibility. Build secure charts. Deploy secure applications. Protect users. Always.

VERIQTA

17. CI/CD with Helm

Helm Production Handbook
Volume 1
Page 17 of 20

Automate everything. Validate always. Rollback fast.
A great pipeline delivers reliably and safely.

Code → Build → Test → Package → Deploy → Validate → Monitor → Rollback (if needed)

1 Automated Deployments

- [x] Trigger on push/merge
- [x] Build and package chart
- [x] Deploy to target env
- [x] Zero manual steps
- [x] Consistent & repeatable

2 Validation at Every Step

- [x] Lint chart
- [x] Template render
- [x] Unit tests (helm-unittest)
- [x] Security scan
- [x] Deploy preview (optional)

3 Rollback Automation

- [x] Detect failed deploys
- [x] Auto rollback on failure
- [x] Rollback to last good rev
- [x] Notify team
- [x] Record incident

4 Environment Promotion

- [x] Dev → Test → Staging
- [x] Approve before prod
- [x] Version controlled
- [x] Audit every promotion
- [x] Gate with quality checks

5 Observability Integration

- [x] Check deployments
- [x] Monitor metrics
- [x] Alert on errors
- [x] Track release health
- [x] Feedback loop

6 GitOps Friendly

- [x] Chart in Git repo
- [x] Values per environment
- [x] Pull request review
- [x] Autonomous sync
- [x] Drift detection

7 Jenkins Integration

- [x] Pipeline as Code
- [x] Strong plugin ecosystem
- [x] Great for complex flows
- [x] Built-in approvals

Jenkinsfile (Example)

```
pipeline {
  agent any
  stages {
    stage('Lint') {
      steps { sh 'helm lint .' }
    }
    stage('Deploy') {
      steps {
sh 'helm upgrade --install myapp ./chart \
  --namespace $NAMESPACE --atomic \
  --timeout 10m'
      }
    }
  }
}
```

8 Github Actions

- [x] Easy to start
- [x] Marketplace actions
- [x] Secure with OIDC
- [x] Great for open source

Workflow (Example)

```
deploy:
  runs-on: ubuntu-latest
  steps:
  - uses: actions/checkout@v4
  - uses: azure/setup-helm@v4
  - run: helm lint .
  - run: helm template .
         --values values-test.yaml
- run: helm upgrade --install myapp ./chart \
    --namespace $NAMESPACE --atomic
```

Secrets

Store in GitHub Secrets
Use OIDC for cloud auth

9 GitLab CI

- [x] Built-in CI/CD
- [x] Auto DevOps support
- [x] Helm chart registry
- [x] Environment dashboards

.gitlab-ci.yml (Example)

```
stages:
  - test
  - deploy

lint:
  stage: test
  script:
  - helm lint .

deploy_prod:
  stage: deploy
  script:
  - helm upgrade --install myapp ./chart \
      --namespace prod --atomic
  environment:
    name: production
    url: https://app.example.com
```

10 Azure Devops

- [x] Enterprise ready
- [x] Environments & approvals
- [x] Variable groups
- [x] Great audit & security

azure-pipelines.yaml (Example)

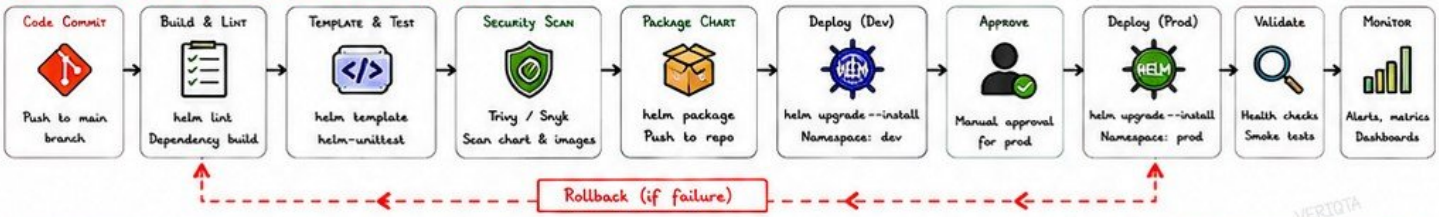
```
trigger:
  branches:
    include: [ main ]

stages:
- stage: Deploy
  jobs:
  - job: DeployProd
    pool:
      vmImage: 'ubuntu-latest'
    steps:
    - task: HelmInstaller@1
    - script: |
        helm upgrade --install myapp ./chart \
          --namespace prod --atomic
```

Service Connections

Use Azure Service Connections
with least privilege access

11 Production Helm Pipeline Workflow (Example)



12 Validation Checklist

- [x] helm lint
- [x] helm template
- [x] helm unittest
- [x] Security scan
- [x] Image scan
- [x] Policy check (OPA/Conftest)
- [x] Kubernetes manifest validation

13 Rollback Automation (Example)

```
if [ "$STATUS" != "success" ]; then
  echo "Deployment failed. Rolling back...."
helm rollback myapp 0 --namespace $NAMESPACE
fi   "Rollback completed."
```

Always use --atomic for safety.

14 Production Best Practices

- [x] Version everything (charts & values)
- [x] Use immutable images
- [x] Use --atomic and --timeout
- [x] Test in a staging environment
- [x] Use Helm repositories (not local only)
- [x] Separate values per environment
- [x] Monitor after every deployment
- [x] Document rollback process

15 Common Pipeline Mistakes

- [x] Skipping helm lint or template
- [x] Hardcoding credentials
- [x] Not using --atomic
- [x] Deploying directly to production
- [x] Missing health checks
- [x] No rollback plan
- [x] Using wrong values file
- [x] Ignoring security scans

Automate the boring. Validate the critical. Deploy with confidence.
A strong pipeline is your safety net in production.

VERIQTA

VERIQTA

HELM PRODUCTION HANDBOOK
VOLUME 1
PAGE 18 OF 20

18. ENTERPRISE HELM ARCHITECTURE

Design Helm for scale, consistency, security, and speed.
One chart. Many environments. Consistent delivery.

Standardize → Reuse → Automate → Secure → Observe

① ENTERPRISE HELM ARCHITECTURE OVERVIEW

DEVELOPERS
- Write code
- Update values
- Commit to Git

→

GIT REPOSITORY
- Charts
- Values
- Environments
- Pipelines

→

CI/CD PIPELINE
- Lint
- Test
- Package
- Push chart
- Deploy

→

HELM REPOSITORY
- OCI Registry
or
Chart Repository
- Versioned Charts

→

KUBERNETES CLUSTERS

DEV
- Dev values
- Lower resources
- Fast iteration

STAGING
- Staging values
- Testing
- Integration

PRODUCTION
- Prod values
- High availability
- Observability

OBSERVABILITY & SECURITY (CROSS-CUTTING)

Prometheus · Grafana · Loki · Alerts · RBAC · Policies

② MULTI-ENVIRONMENT STRATEGY

ENVIRONMENT	PURPOSE	VALUES FILE	CLUSTER	RESOURCE LEVEL
DEV	Development & feature work	values-dev.yaml	dev-cluster	Low
STAGING	Testing & verification	values-staging.yaml	staging-cluster	Medium
PRODUCTION	Live traffic & business	values-prod.yaml	prod-cluster	High

Promote the same chart across environments using different values.

③ SHARED CHARTS & COMPONENTS

SHARED LIBRARY CHARTS
- common (labels, annotations, helpers)
- logging (Fluent Bit, Loki agent)
- monitoring (Prometheus exporters)
- security (Pod Security, PSP, RBAC)
- networking (Ingress, Service)
- database (PostgreSQL, Redis)
- autoscaling (HPA, VPA)

APPLICATION CHARTS
- user-service
- order-service
- payment-service
- frontend-web
- notification-service
- batch-jobs

Build once. Reuse everywhere. Keep it DRY.

④ VERSIONING STRATEGY

SEMANTIC VERSIONING

MAJOR.MINOR.PATCH

1.4.2

- 1 — Breaking Changes
- 4 — New Features Backward Compatible
- 2 — Bug Fixes Backward Compatible

BEST PRACTICES
- ☑ Follow SemVer strictly
- ☑ Never change existing versions
- ☑ Deprecate before removal
- ☑ Document all changes
- ☑ Use CHANGELOG.md
- ☑ Tag releases in Git
- ☑ Sign charts for integrity

⑤ GITOPS PREPARATION WITH HELM

GIT REPO STRUCTURE (EXAMPLE)

```
helm-apps/
├── charts/
│   ├── myapp/
│   └── common/
└── environments/
    ├── dev/
    │   └── values.yaml
    ├── staging/
    │   └── values.yaml
    └── prod/
        └── values.yaml
├── charts/values.schema.json
└── README.md
```

GITOPS TOOLS
- Argo CD
- Flux CD
- Helm + Kustomize
- Weave GitOps

GITOPS BENEFITS
- ☑ Declarative
- ☑ Version controlled
- ☑ Auditable
- ☑ Rollbacks are easy
- ☑ Drift detection
- ☑ Self-healing
- ☑ Consistency

Git is the source of truth. Clusters are just the execution.

⑥ ENTERPRISE RECOMMENDATIONS

- ☑ Standardize chart structure and conventions
- ☑ Use values.schema.json for validation
- ☑ Enforce code review for all changes
- ☑ Scan charts for vulnerabilities
- ☑ Use RBAC and least privilege
- ☑ Sign charts and verify provenance
- ☑ Monitor everything
- ☑ Automate rollbacks
- ☑ Document everything
- ☑ Train your team

ENTERPRISE CHECKLIST
- ☑ Chart linting in CI
- ☑ Unit tests for templates
- ☑ Security scanning
- ☑ Helm chart signing
- ☑ Version policy enforced
- ☑ Promotion workflow
- ☑ Observability enabled
- ☑ Disaster recovery tested

⑦ ENTERPRISE HELM BEST PRACTICES

STANDARDIZE	AUTOMATE	SECURE	OBSERVE	DOCUMENT	CONTINUOUS IMPROVEMENT
One standard chart structure	CI/CD & GitOps	Least privilege & signed charts	Monitor & alert	Docs, runbooks & guides	Review, learn & evolve

Great architecture is invisible. Great Helm architecture scales quietly and delivers reliably.

VERIQTA

19. Production Case Studies

Helm Production Handbook
Volume 1
Page 19 of 20

Real-world deployments. Real challenges. Real lessons.
Learn from what works — and what breaks.

Every case teaches you how to deploy, troubleshoot, and operate Helm in production.

1 NGINX Deployment

- Using official NGINX Ingress chart
- Custom values for IngressClass
- TLS termination with cert-manager
- HPA for traffic spikes
- Anti-affinity for HA

Key Config

```
controller:
  replicaCount: 3
service:
  type: LoadBalancer
metrics:
  enabled: true
```

Outcome: High availability, secure, scalable ingress.

2 Prometheus Deployment

- Using kube-prometheus-stack
- Persistent storage for TSDB
- Retention and resource tuning
- Alertmanager integration
- ServiceMonitor autodiscovery

Key Config

```
prometheus:
  prometheusSpec:
    retention: 15d
    storageSpec:
      volumeClaimTemplate: {...}
```

Outcome: Reliable metrics, alerting, retention.

3 Grafana Deployment

- Official Grafana chart
- Admin user via secret
- Dashboards via ConfigMap
- Datasource provisioning
- HA with multiple replicas

Key Config

```
adminUser: admin
persistence:
  enabled: true
dashboardsProvider:
  enabled: true
```

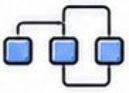
Outcome: Centralized dashboards and observability.

4 Microservices Deployment

- Multiple microservices via Helm
- Each service with its chart
- Shared library chart for common logic
- Ingress per service
- Resource quotas and limits

Key Practices

- ☑ Reusable templates
- ☑ Consistent labels
- ☑ Global values for environment
- ☑ Independent deployments



Outcome: Scalable, modular, maintainable services.

5 Database Deployment

- Bitnami PostgreSQL chart
- Persistent storage
- Backup with cronjob
- Read replicas for scale
- Secrets for credentials

Key Config

```
auth:
  postgresPassword: <secret>
primary:
  persistence:
    size: 50Gi
```

Outcome: Durable, secure, and backed up.

6 Other Common Charts

- → Redis: caching and session store
- → Kafka: messaging backbone
- → Elasticsearch: log storage & search
- → RabbitMQ: message queue
- → Keycloak: identity and SSO

All deployed with Helm, customized for production needs.

Outcome: Consistent deployments, fewer mistakes.

7 Common Production Failures

- ✗ Incorrect values.yaml
  Wrong resource limits, replica counts, or configs.
- ✗ Missing persistent storage
  Data lost after pod restart or upgrade.
- ✗ No resource requests/limits
  Causes throttling or OOMKills.
- ✗ Ignoring dependencies
  Subcharts or CRDs not installed.
- ✗ Hardcoded secrets
  Secrets exposed in git or values files.
- ✗ Upgrades without testing
  Breaking changes not caught early.
- ✗ No health checks
  Pods appear healthy but app is not.

Impact

- Downtime
- Data loss
- Performance issues
- Security risks
- Escalation at 2AM

8 Senior Engineer Lessons

- ☑ Always test in a safe environment first.
- ☑ Read the chart README and values.
- ☑ Understand what the chart deploys.
- ☑ Use lint, template, and diff before deploy.
- ☑ Monitor everything after deployment.
- ☑ Plan for rollback before upgrading.
- ☑ Keep charts simple, values separated.
- ☑ Document your customizations.

Helm is easy to start.
Hard to master.
Production makes you better.

9 Production Best Practices

- ☑ Use official or trusted charts.
- ☑ Pin chart versions.
- ☑ Store values in Git.
- ☑ Use environments: dev, staging, prod.
- ☑ Validate with: lint, template, kubeval.
- ☑ Enable RBAC and least privilege.
- ☑ Encrypt secrets and use External Secrets.
- ☑ Automate with CI/CD and GitOps.
- ☑ Monitor, alert, and log everything.
- ☑ Keep backups and disaster recovery.

Golden Rule:
Deploy like a pro.
Operate like a SRE.
Sleep like a baby.

Production Deployment Mindset

Plan — Understand requirements → Design — Choose the right chart → Customize — Values & templates → Validate — Lint, template, security → Deploy — Automate & monitor → Observe — Metrics, alerts, logs → Improve — Learn, iterate, harden

Great deployments aren't lucky.
They are designed, tested, and earned.
VERIQTA

VERIQTA

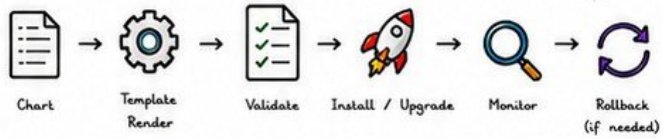
20. Volume 1 Production Review

Helm Production Handbook
Volume 1
Page 20 of 20

Everything you learned. Everything you need.
One page to remember it all. Build. Ship. Operate. Repeat.

Helm turns Kubernetes complexity into repeatable, reliable, and production-ready operations.

1 Helm Workflow (The Big Picture)



Core Principles

- ☑ Declarative
- ☑ Repeatable
- ☑ Versioned
- ☑ Portable
- ☑ Reliable

Build once.
Deploy everywhere.
Operate with confidence.

2 Helm Command Cheat Sheet

Install / Upgrade / Uninstall

```
helm repo add <name> <url>          # add repo
helm repo update                    # update repos
helm search repo <chart>            # search charts
helm install <rel> <chart> \
  -n <ns> -f values.yaml            # install
helm upgrade <rel> <chart> \
  -n <ns> -f values.yaml            # upgrade
helm uninstall <rel> -n <ns>        # uninstall
```

Templating & Validation

```
helm template <rel> <chart> -f values.yaml \
  --debug                           # render locally
helm lint <chart>                   # lint chart
helm install <rel> <chart> -f values.yaml \
  --dry-run --debug                 # dry run
```

Inspect & Debug

```
helm list -A                        # list releases
helm status <rel> -n <ns>           # release status
helm get all <rel> -n <ns>          # all info
helm get values <rel> -n <ns>       # user values
helm get manifest <rel> -n <ns>     # rendered manifests
helm describe <rel> -n <ns>         # details & events
```

Rollback & History

```
helm history <rel> -n <ns>          # revision history
helm rollback <rel> <rev> -n <ns>   # rollback
helm diff upgrade <rel> <chart> \
  -f values.yaml                    # diff changes
```

Hooks & Tests

```
helm test <rel> -n <ns>             # run tests
helm get hooks <rel> -n <ns>        # list hooks
```

3 Production Deployment Checklist

Before Deployment

- ☑ Chart version reviewed
- ☑ Values validated
- ☑ Dependencies updated
- ☑ Lint passed
- ☑ Security scan passed

During Deployment

- ☑ Use --atomic
- ☑ Use --timeout
- ☑ Namespace exists
- ☑ Resource limits set
- ☑ Readiness probes set

After Deployment

- ☑ Release status healthy
- ☑ Pods running & ready
- ☑ Service endpoints OK
- ☑ Metrics & alerts active
- ☑ Smoke tests passed

4 Common Mistakes

- ✗ Hardcoding values in templates
- ✗ Not using values.yaml
- ✗ Missing resource limits
- ✗ Ignoring chart version updates
- ✗ Installing in wrong namespace
- ✗ Overriding critical values
- ✗ Not using --atomic
- ✗ Not testing rollback
- ✗ Large charts without validation
- ✗ Storing secrets in values.yaml

Small mistakes in Helm become big incidents in production.

5 Troubleshooting Checklist

- ☑ helm status <rel> -n <ns>
- ☑ helm describe <rel> -n <ns>
- ☑ Check Kubernetes events
- ☑ helm get manifest <rel> -n <ns>
- ☑ helm template <rel> <chart> --debug
- ☑ Check hooks and jobs
- ☑ Check resource limits & quotas
- ☑ Check pods, PVCs, services, endpoints
- ☑ Check logs of failing pods
- ☑ Rollback if impact is high

6 Security Checklist

- ☑ Use signed or verified charts
- ☑ Store secrets in External Secrets
- ☑ Encrypt secrets at rest
- ☑ RBAC least privilege
- ☑ No root containers
- ☑ Scan images for vulnerabilities
- ☑ Pin chart & image versions
- ☑ Network policies enabled
- ☑ Limit service account access
- ☑ Audit Helm releases

7 Senior Engineer Insights

- ★ Helm is not just for installation, it's for OPERATIONS.
- ★ Your charts are part of your application code.
- ★ Consistency across environments is everything.
- ★ Values drive behavior. Manage them like code.
- ★ Observability makes Helm safe in production.
- ★ Rollback is a feature. Test it before you need it.
- ★ Automate everything. Manual steps create incidents.
- ★ Security is not optional. Build it in from day one.
- ★ Good charts are simple, modular, and well-documented.
- ★ Great engineers build systems that heal and scale.

8 Quick Production Reference

Install

```
helm install
<rel> <chart>
-n <ns> -f values.yaml
--atomic --timeout 10m
```

Upgrade

```
helm upgrade
<rel> <chart>
-n <ns> -f values.yaml
--atomic
```

Rollback

```
helm rollback
<rel> <revision>
-n <ns>
```

Uninstall

```
helm uninstall
<rel> -n <ns>
```

Status

```
helm status
<rel> -n <ns>
```

Manifest

```
helm get manifest
<rel> -n <ns>
```

Values

```
helm get values
<rel> -n <ns> -a
```

History

```
helm history
<rel> -n <ns>
```

Helm makes Kubernetes simple. You make it production-ready.
Build smart. Deploy safe. Operate with confidence.